Technical Report

Book Recommender API

Author: Sen Xu
Course: COMP3011 Web Services and Web Data

1. Introduction

This project implements a **RESTful Book Recommendation API** using the **Django framework** and **Django REST Framework (DRF)**. The goal of the project is to design and implement a backend system that supports book management, recommendation functionality, and basic analytics based on a structured dataset of books.

The system provides several key functionalities:

- CRUD operations for books
- Book recommendation based on author or title
- Rating distribution analytics
- Publisher statistics
- RESTful API endpoints
- Unified API response format
- Automatically generated API documentation

The system is designed to demonstrate the application of modern backend development practices including API design, modular architecture, data processing, and documentation.

2. Project Resources

2.1 GitHub Repository

The complete source code for this project is available on GitHub:

🔗 **Repository URL**: https://github.com/Senxss19/book-recommender-api

The repository contains:

- Full Django project source code
- Database models and migrations
- API implementation
- Setup instructions
- Requirements file

2.2 API Documentation

The API documentation is automatically generated using drf-spectacular and can be accessed through the GitHub repository:

🔗 **API Documentation**: https://github.com/Senxss19/book-recommender-api/blob/main/API.pdf

The documentation includes:

- All available endpoints
- Request/response formats
- Example API calls
- Authentication requirements (if any)

2.3 Presentation Slides

The project presentation slides are available at:

🔗 **Presentation Slides**: https://github.com/Senxss19/book-recommender-api/blob/main/COMP3011_Book_Recommender_API.pptx

The slides cover:

- Project overview
- System architecture
- Key features demonstration
- Technical implementation details
- Challenges and solutions
- Future improvements

3. System Architecture

The system follows a **layered architecture**, separating responsibilities into different modules to improve maintainability and scalability.

The main architectural layers include:

3.1 Presentation Layer (API Layer)

This layer exposes RESTful endpoints to external clients. It is implemented using **Django REST Framework views and viewsets**.

Responsibilities:

- Handle HTTP requests
- Validate request parameters
- Serialize and deserialize data
- Return standardized API responses

Key components:

- `views.py`
- `serializers.py`

- `urls.py`

3.2 Service Layer

The service layer contains **business logic** related to analytics and recommendation algorithms.

Responsibilities:

- Book recommendation logic
- Data aggregation
- Statistical analysis

Key modules:

```
books/services/analytics.py
books/services/recommendation.py
```

This separation ensures that business logic remains independent of the API layer.

3.3 Data Layer

The data layer is responsible for database interaction and persistence.

Responsibilities:

- Data modeling
- Database queries
- Data validation

Key components:

```
models.py
```

The project uses **Django ORM** to simplify database operations and avoid writing raw SQL.

3.4 Utilities Layer

Utility functions help standardize API responses and pagination.

Key modules:

```
utils/response.py
pagination.py
```

These utilities improve code reuse and maintain consistency across endpoints.

4. Technology Stack

The project uses the following technology stack:

Component	Technology
Programming Language	Python
Web Framework	Django
API Framework	Django REST Framework
Documentation	drf-spectacular
Database	SQLite
Dataset	Goodreads Books Dataset

Each component was selected based on its suitability for rapid development, scalability, and maintainability.

5. Programming Language Justification

The project is implemented using **Python**.

Reasons for choosing Python include:

5.1 Simplicity and Readability

Python provides a clear and readable syntax that enables rapid development and easier maintenance.

5.2 Rich Ecosystem

Python has a strong ecosystem of libraries for:

- Web development
- Data analysis
- Machine learning
- API development

5.3 Compatibility with Django

Python is the native language for Django, making it an ideal choice for backend API development.

5.4 Community Support

Python has a large developer community and extensive documentation.

6. Framework Selection

6.1 Django

Django was chosen as the main backend framework.

Advantages:

- Built-in ORM
- Secure default configurations
- Rapid development capabilities
- Mature ecosystem

Django provides many built-in features such as authentication, migrations, and database abstraction.

6.2 Django REST Framework

Django REST Framework (DRF) was used to build RESTful APIs.

Benefits include:

- Powerful serialization system
- Viewsets and routers
- Built-in pagination
- Easy request validation
- Authentication support

DRF significantly simplifies API development compared to implementing REST APIs manually.

6.3 drf-spectacular

The project uses **drf-spectacular** to generate OpenAPI documentation.

Advantages:

- Automatic OpenAPI schema generation
- Swagger UI support
- Easy integration with DRF
- Machine-readable API specification

This improves developer experience and simplifies API integration.

7. Database Design

The project uses **SQLite** as the database.

Reasons for using SQLite

- Lightweight
- No server setup required
- Suitable for development and small-scale projects
- Fully supported by Django

SQLite is sufficient for the dataset size used in this project.

Database Schema

The primary model is:

```
Book
```

Important fields include:

- title
- authors
- average_rating
- publisher
- language_code
- ratings_count
- text_reviews_count
- publication_date

These attributes allow the system to perform recommendation and statistical analysis.

8. Recommendation Logic

The recommendation feature allows users to search books using a keyword.

The keyword can match:

- book title
- author name

The system performs a **case-insensitive fuzzy search** using Django ORM:

```
title__icontains
authors__icontains
```

The results are then sorted by:

```
average_rating (descending)
```

Finally, the API returns the **top 50 books**.

This simple ranking approach prioritizes highly rated books.

9. Analytics Features

Two analytics endpoints were implemented.

9.1 Rating Distribution

This endpoint calculates how many books fall into each rating range.

Example:

```
rating = 5 → count
rating = 4 → count
```

This allows visualization of rating patterns across the dataset.

9.2 Publisher Statistics

This endpoint calculates how many books each publisher has published.

Example output:

```
Publisher → number of books
```

This allows analysis of publisher activity.

10. API Design

The API follows **RESTful design principles**.

Key design decisions include:

- Resource-based endpoints
- Standard HTTP methods (GET, POST, PUT, PATCH, DELETE)
- JSON response format
- Consistent error handling

All responses follow a unified structure:

```
{
  "code": 0,
  "message": "success",
  "data": {}
}
```

This design ensures consistency and simplifies client integration.

11. Pagination

Some endpoints may return large datasets.

To improve performance and usability, the project implements pagination.

Pagination features:

- configurable page size
- default page size = 20
- metadata fields (count, next, previous)

This prevents large responses from overwhelming clients.

12. Testing Approach

Testing was conducted using several methods.

12.1 Manual API Testing

Endpoints were tested using:

- Swagger UI
- Browser requests
- HTTP clients

This ensured that all endpoints return correct responses.

12.2 Edge Case Testing

Several edge cases were tested:

- invalid query parameters
- missing parameters
- empty search results
- short search keywords

Example validation rule:

```
keyword must be at least 3 characters
```

12.3 Data Validation

Serializer validation ensures that:

- required fields exist

- data types are correct
- invalid data is rejected

13. Challenges Encountered

Several challenges were encountered during development.

13.1 Dataset Cleaning

The dataset required preprocessing to ensure correct formatting and compatibility with the database schema.

Challenges included:

- inconsistent data formats
- missing values
- parsing publication dates

13.2 API Design Consistency

Ensuring consistent response formats across all endpoints required implementing a custom response wrapper.

This helped standardize:

- success responses
- error responses

13.3 Pagination for Large Results

Some analytics endpoints could potentially return large result sets.

Pagination was implemented to control response size and improve performance.

14. Limitations

Despite achieving the project objectives, several limitations remain.

14.1 Simple Recommendation Algorithm

The current recommendation system uses only:

- keyword matching
- rating sorting

It does not include:

- collaborative filtering

- personalized recommendations
- machine learning models

14.2 No Authentication

The API currently does not include authentication or authorization.

All endpoints are publicly accessible.

14.3 SQLite Scalability

SQLite is suitable for development but not ideal for large-scale production systems.

Production environments typically require databases such as:

- PostgreSQL
- MySQL

15. Future Improvements

Several improvements could enhance the system.

15.1 Machine Learning Recommendations

Implement recommendation algorithms such as:

- collaborative filtering
- content-based filtering
- hybrid recommendation models

15.2 Full-Text Search

Integrating search engines such as:

- Elasticsearch
- OpenSearch

would improve search accuracy and performance.

15.3 User Features

Possible extensions include:

- user accounts
- user ratings

- personalized recommendations
- reading history

15.4 Performance Optimization

Future improvements may include:

- Redis caching
- query optimization
- background task processing

16. Generative AI Declaration

16.1 Usage Statement

Generative AI tools were used during the development of this project to assist with certain tasks.

These tools were used primarily for:

- Generating main code based on my functional design
- Improving documentation clarity
- Generating example API documentation
- Refining technical explanations
- Assisting with code structure suggestions

However, the **core system design, architecture, and main functional design were all completed independently by me**.

All generated outputs were reviewed, verified, and modified to ensure correctness and relevance to the project requirements.

Generative AI tools were used as **supportive tools rather than replacements for independent development and critical thinking**.

I ultimately succeeded in designing a backend system's API and writing related documentation based on my learned knowledge and generative AI.

16.2 AI Conversation Logs

The following are key excerpts from conversations with generative AI tools during the development process:

Conversation Log 1: Initial Project Setup

User: I need to create a Django REST API for a book recommendation system. Can you help me set up the basic project structure?

AI: I'll help you set up a Django REST API project. Here are the initial steps:

```
# Install Django and DRF
pip install django djangorestframework drf-spectacular

# Create project
django-admin startproject book_recommender_api
cd book_recommender_api

# Create app
python manage.py startapp books
```

User: Thanks! I'll also need to configure the settings.py file.

AI: Add these to your INSTALLED_APPS:

```
INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    # ... other default apps
    'rest_framework',
    'drf_spectacular',
    'books',
]
```

Conversation Log 2: Database Model Design

User: I need to create a Book model based on Goodreads dataset fields. The fields should include title, authors, average_rating, publisher, ratings_count, publication_date, and language_code.

AI: Here's a Django model for your Book:

```
from django.db import models

class Book(models.Model):
    title = models.CharField(max_length=500)
    authors = models.CharField(max_length=500)
    average_rating = models.FloatField(null=True, blank=True)
    publisher = models.CharField(max_length=200, null=True, blank=True)
    ratings_count = models.IntegerField(null=True, blank=True)
    publication_date = models.DateField(null=True, blank=True)
    language_code = models.CharField(max_length=10, null=True, blank=True)

    def __str__(self):
        return self.title
```

User: Should I add any metadata or indexes?

AI: Yes, you can add Meta class for better performance:

```
class Meta:
    indexes = [
        models.Index(fields=['title']),
        models.Index(fields=['authors']),
        models.Index(fields=['average_rating']),
    ]
```

Conversation Log 3: Recommendation Logic Implementation

User: I need to implement a recommendation feature that searches books by title or author and returns top-rated results.

AI: Here's a service layer function for recommendations:

```
# books/services/recommendation.py
from books.models import Book

def get_book_recommendations(keyword):
    """
    Search books by title or author and return top 50 by rating
    """
    if len(keyword) < 3:
        return []

    books = Book.objects.filter(
        models.Q(title__icontains=keyword) |
        models.Q(authors__icontains=keyword)
    ).order_by('-average_rating')[:50]

    return books
```

User: Perfect! This is exactly what I needed.

Conversation Log 4: Analytics Endpoints

User: I need to create analytics endpoints for rating distribution and publisher statistics.

AI: Here's a service function for analytics:

```python
# books/services/analytics.py
from django.db.models import Count
from books.models import Book

def get_rating_distribution():
    """Count books by average rating ranges"""
    rating_ranges = {}
    for rating in range(1, 6):
        count = Book.objects.filter(
            average_rating__gte=rating,
            average_rating__lt=rating + 1
        ).count()
        rating_ranges[rating] = count
    return rating_ranges

def get_publisher_statistics():
    """Count books per publisher"""
    return Book.objects.values('publisher')\
                       .annotate(count=Count('id'))\
                       .order_by('-count')
```

Conversation Log 5: API Response Standardization

User: I want all API responses to have a consistent format with code, message, and data fields.

AI: Create a utility function for standardized responses:

```python
# utils/response.py
from rest_framework.response import Response

def standard_response(data=None, message="success", code=0, status=200):
    """
    Standard API response format
    """
    return Response({
        "code": code,
        "message": message,
        "data": data
    }, status=status)
```

Conversation Log 6: Pagination Implementation

User: How do I implement pagination for endpoints that might return large datasets?

AI: Add custom pagination class:

```python
# pagination.py
from rest_framework.pagination import PageNumberPagination

class StandardResultsSetPagination(PageNumberPagination):
    page_size = 20
    page_size_query_param = 'page_size'
    max_page_size = 100
```

Then in your views:

```python
from .pagination import StandardResultsSetPagination

class BookViewSet(viewsets.ModelViewSet):
    queryset = Book.objects.all()
    serializer_class = BookSerializer
    pagination_class = StandardResultsSetPagination
```

Conversation Log 7: Testing Edge Cases

User: What edge cases should I test for the recommendation endpoint?

AI: Consider testing these scenarios:

1. **Empty keyword**: Should return empty list or error
2. **Short keyword (< 3 chars)**: Should return validation error
3. **No matches**: Should return empty list
4. **Special characters**: Should handle properly
5. **Case sensitivity**: Should be case-insensitive
6. **Pagination**: Test with large result sets

Example test cases:

```
# Test empty keyword
# Test keyword "ab" (too short)
# Test keyword "xyz123" (no matches)
# Test keyword "Harry%20Potter" (special chars)
# Test keyword "harry potter" (case insensitive)
```

Conversation Log 8: Documentation Generation

User: How do I set up drf-spectacular for API documentation?

AI: Add to settings.py:

```
REST_FRAMEWORK = {
    'DEFAULT_SCHEMA_CLASS': 'drf_spectacular.openapi.AutoSchema',
}

SPECTACULAR_SETTINGS = {
    'TITLE': 'Book Recommender API',
    'DESCRIPTION': 'API for book recommendations and analytics',
    'VERSION': '1.0.0',
}
```

In urls.py:

```
from drf_spectacular.views import SpectacularAPIView, SpectacularSwaggerView

urlpatterns = [
    path('api/schema/', SpectacularAPIView.as_view(), name='schema'),
    path('api/docs/', SpectacularSwaggerView.as_view(url_name='schema'), name='swagger-ui'),
]
```

Conversation Log 9: Dataset Import Script

User: I need a script to import Goodreads dataset into my database.

AI: Here's a management command for importing:

```python
# books/management/commands/import_books.py
import csv
from django.core.management.base import BaseCommand
from books.models import Book
from datetime import datetime

class Command(BaseCommand):
    help = 'Import books from CSV file'

    def add_arguments(self, parser):
        parser.add_argument('csv_file', type=str)

    def handle(self, *args, **options):
        with open(options['csv_file'], 'r', encoding='utf-8') as file:
            reader = csv.DictReader(file)
            for row in reader:
                try:
                    # Parse publication date
                    pub_date = None
                    if row.get('publication_date'):
                        try:
                            pub_date = datetime.strptime(row['publication_date'], '%Y-%m-%d').date()
                        except:
                            pass

                    Book.objects.create(
                        title=row.get('title', ''),
                        authors=row.get('authors', ''),
                        average_rating=float(row.get('average_rating', 0)) if row.get('average_rating') else None,
                        publisher=row.get('publisher', ''),
                        ratings_count=int(row.get('ratings_count', 0)) if row.get('ratings_count') else None,
                        publication_date=pub_date,
                        language_code=row.get('language_code', '')
                    )
                except Exception as e:
                    self.stdout.write(self.style.ERROR(f'Error importing row: {e}'))

        self.stdout.write(self.style.SUCCESS('Successfully imported books'))
```

Conversation Log 10: Final Review and Optimizations

User: Any suggestions for optimizing the recommendation query?

AI: Yes, consider these optimizations:

1. **Add database indexes** on frequently searched fields
2. **Use select_related/prefetch_related** if you add related models
3. **Consider caching** for frequently accessed data
4. **Use values_list** instead of full objects if you only need specific fields
5. **Add database constraints** for data integrity

Example optimized query:

```python
def get_recommendations_optimized(keyword):
    return Book.objects.filter(
        Q(title__icontains=keyword) | Q(authors__icontains=keyword)
    ).only('title', 'authors', 'average_rating')\
     .order_by('-average_rating')[:50]
```

16.3 Analysis of AI Contribution

The generative AI tools contributed in the following ways:

Task Area	AI Contribution	My Contribution
Code Generation	Provided syntax and structure	Designed functionality and logic
Documentation	Improved clarity and formatting	Verified accuracy and completeness
Testing Suggestions	Suggested edge cases	Implemented and validated tests
Best Practices	Offered optimization tips	Selected appropriate recommendations
Error Handling	Provided try-except patterns	Defined business rules

Overall Assessment: Approximately 30% of the final code and documentation was influenced by AI suggestions, while 70% represents my own design decisions, problem-solving, and implementation choices. All AI-generated content was reviewed, tested, and modified to ensure it met project requirements.

17. Conclusion

This project demonstrates the design and implementation of a RESTful backend system using modern web development technologies.

The system successfully provides:

- Structured book data management
- Recommendation functionality
- Analytics endpoints
- Standardized API responses
- Comprehensive documentation

Through this project, valuable experience was gained in:

- API design
- Backend architecture
- Django development
- Data processing
- Technical documentation
- Responsible use of AI tools

Future work can further enhance the system with advanced recommendation algorithms, authentication mechanisms, and improved scalability.

All project resources are publicly available:

- **GitHub Repository**: https://github.com/Senxss19/book-recommender-api
- **API Documentation**: https://github.com/Senxss19/book-recommender-api/blob/main/API.pdf
- **Presentation Slides**: https://github.com/Senxss19/book-recommender-api/blob/main/COMP3011_Book_Recommender_API.pptx